POO

Programação Orientada a Objetos

Recapitulando os pilares anteriores

- Classes, métodos, atributos
- Encapsulamento
- Herança

O que é Polimorfismo?

- Definição simples: “Muitas formas”
- Uma mesma interface pode ter comportamentos diferentes dependendo do objeto.
- Exemplo cotidiano: Um botão "Imprimir" que funciona com impressora HP, Epson, etc.

Polimorfismo com métodos sobrescritos

```
class Animal:
    def falar(self):
        print("O animal faz um som")

class Cachorro(Animal):
    def falar(self):
        print("Au au!")

class Gato(Animal):
    def falar(self):
        print("Miau!")
```

Polimorfismo em ação (lista de objetos)

```
animais = [Cachorro(), Gato()]

for animal in animais:
    animal.falar()  # Cada um se comporta de forma diferente!
```

Abstração, o que é?

- Definição: Ocultar a complexidade e expor apenas o essencial.
- Exemplo real: Você dirige um carro sem saber como funciona o motor.
- Em código: criar uma classe base genérica, com métodos que serão definidos nas subclasses.

Abstração, o que é?

Usamos abstração quando criamos uma classe genérica, que define um conjunto de métodos que devem ser implementados, mas sem dizer como.

Isso é feito com classes abstratas e métodos abstratos.

Abstração com classes abstratas

```
from abc import ABC, abstractmethod

class Forma(ABC):  # Classe abstrata
    @abstractmethod
    def calcular_area(self):
        pass
```

Abstração com classes abstratas

```
class Retangulo(Forma):
    def __init__(self, largura, altura):
        self.largura = largura
        self.altura = altura

    def calcular_area(self):
        return self.largura * self.altura
```

➡️ **Forma** não pode ser instanciada diretamente
➡️ Toda classe que herda de **Forma** deve obrigatoriamente implementar **calcular_area()**

Por que usar Abstração?

- Força um padrão: garante que todas as subclasses sigam a mesma estrutura.
- Torna o sistema extensível: você pode adicionar novos tipos (Triangulo, Trapézio...) sem mexer no código original.
- Trabalha bem com polimorfismo: permite tratar diferentes objetos como se fossem do mesmo tipo.

Por que usar Abstração?

```
formas = [Retangulo(3, 4), Circulo(5)]

for forma in formas:
    print(forma.calcular_area())  # Comportamento diferente, chamada igual
```

Resumindo...

Polimorfismo permite usar diferentes objetos de forma intercambiável com o mesmo método.

Abstração oculta detalhes da implementação, mostrando apenas o necessário.